

Cahier des Modifications — Backend

Modifications réalisées après la version documentée dans `AGENTS.md` (session août 2026). Branche : `Backend` — déploiement auto sur Railway.

1. Cycle de vie complet des litiges escrow

1.1 Migration `20260814002635_escrow_dispute_resume`

Deux colonnes ajoutées à `EscrowTransaction` :

Colonne	Type	Rôle
<code>status_before_dispute</code>	<code>String?</code> <code>@db.VarChar(30)</code>	Statut de la vente au moment du litige (pour la reprise)
<code>disputed_by</code>	<code>Int?</code>	Auteur du litige (<code>buyer_id</code> ou <code>seller_id</code>) — se lui ou l'admin peut refermer

1.2 Ouverture d'un litige — `escrow.service.js` → `disputeEscrow`

- Enregistre désormais `status_before_dispute: escrow.status` et `disputed_by: userId` en plus de `status: 'disputed', disputed_at, dispute_reason`.
- Interdit après `completed` / `cancelled` / `refunded` (400).
- Réserve à l'acheteur ou au vendeur (403 sinon).

1.3 Format exposé — `formatEscrow`

- Nouveau champ retourné : `disputedBy (escrow.disputed_by ?? null)` → utilisé côté frontend pour n'afficher le bouton « Reprendre la vente » qu'à l'auteur du litige.

1.4 Reprise de la vente — `escrow.service.js`

Fonction	Visibilité	Logique
<code>restoreFromDispute(id)</code> (interne)	—	Verrou atomique <code>updateMany({ id, status: 'disputed' })</code> → restaure <code>status_before_dispute</code> , efface <code>status_before_dispute, disputed_by, disputed_at, dispute_reason</code> . Erreurs : 404 introuvable, 400 déjà traité, 400 si statut d'origine hors liste blanche.
<code>cancelDispute(id, userId)</code>	Utilisateur	403 si <code>disputed_by !== userId</code> (« Seul l'auteur du litige peut le refermer ») → <code>restoreFromDispute</code>
<code>resumeDispute(id)</code>	Admin	<code>restoreFromDispute</code> direct

Liste blanche de reprise : uniquement `paid, pending_delivery, delivered`. Un litige ouvert sur un statut hors liste (ou pré-migration, `status_before_dispute` null) → erreur propre : « Ce litige ne peut pas être repris automatiquement, contactez l'administrateur ».

1.5 Routes ajoutées

Méthode	Path	Auth	Description
PUT	<code>/api/escrow/:id/cancel-dispute</code>	✓ (utilisateur)	Refermer son propre litige → vente reprise
PUT	<code>/api/admin/escrow/:id/resume</code>	✓ (admin)	Reprendre la vente après litige

1.6 Notifications & socket

- `cancel-dispute` (utilisateur) : notifie l'autre partie avec le type `escrow_dispute_cancelled` + émission socket `escrow_updated`.
- `resume` (admin) : notifie l'acheteur **et** le vendeur (même type `escrow_dispute_cancelled`) + `escrow_updated`.

1.7 Résolution admin (existante, complétée) — `admin.controller.js` → `resolveEscrowDispute`

PUT `/api/admin/escrow/:id/resolve` — body { `action: "refund" | "complete"` } :

- **refund** : remboursement de l'acheteur (montant + frais), statut `refunded`.
- **complete** : payout au vendeur (`amount - fee`), statut `completed`.
- Notifications `dispute_resolved` aux deux parties + email au vendeur (html, décision expliquée).
- `escrow.service.js` → `resolveDispute` : refuse si la transaction n'est pas `disputed`.

2. Notifications admin & emails — 73573ad

- À l'**ouverture d'un litige** : notification temps réel à tous les admins (`escrow_updated` + notification type `escrow_dispute_opened`) + **email au vendeur** (Resend, template HTML).
- À la **résolution** : email au vendeur (voir §1.7).

3. Logo du site — 253642a

- Nouvelle clé `site_logo` (URL) dans `SiteSetting` (via PUT `/api/admin/settings`, exposé dans GET `/api/settings/public` et la config admin `admin.service.js` → `DEFAULT_SETTINGS`).
- Frontend : `shared/ui/Logo.jsx` utilise `siteLogo` avec repli sur `assets/logo-tg.png`.

4. Garde « produit vendu » — ba8069e

`admin.service.js` → `updateProductStatus` :

```
if (product.status === 'sold' && status === 'active') {
  // 400 « Un produit vendu ne peut pas être remis en vente »
}
```

Applique aussi côté utilisateur (PUT `/products/:id/status`). Évite la réapparition d'un produit vendu dans les résultats.

5. Fichiers modifiés

Fichier	Contenu
<code>prisma/schema.prisma</code>	+ <code>status_before_dispute</code> , <code>disputed_by</code>

Fichier	Contenu
<code>prisma/migrations/20260814002635_escrow_dispute_resume/migration.sql</code>	Migration additive (aucune donnée détruite)
<code>src/modules/escrow/escrow.service.js</code>	<code>disputeEscrow,</code> <code>formatEscrow,</code> <code>restoreFromDispute,</code> <code>cancelDispute,</code> <code>resumeDispute,</code> <code>resolveDispute</code>
<code>src/modules/escrow/escrow.controller.js</code>	<code>cancelDispute</code> (notifications + socket)
<code>src/modules/escrow/escrow.routes.js</code>	+ <code>PUT /:id/cancel-dispute</code>
<code>src/modules/admin/admin.controller.js</code>	<code>resolveEscrowDispute,</code> <code>resumeEscrow</code>
<code>src/modules/admin/admin.routes.js</code>	+ <code>PUT /escrow/:id/resolve,</code> <code>PUT /escrow/:id/resume</code>
<code>src/modules/admin/admin.service.js</code>	<code>updateProductStatus</code> (garde sold), <code>DEFAULT_SETTINGS.site_logo</code>

6. Tests effectués (API live Railway)

Test	Résultat
Connexion socket avec token	OK (<code>CONNECTE : ...</code>)
<code>join_room</code> + envoi message REST → <code>new_message</code> reçu	OK
Deux sockets (2 comptes) : émission socket + REST → réception croisée	OK (A 2/2, B 2/2)
<code>PUT /admin/escrow/999999/resume</code> (inexistant)	404 « Transaction introuvable » (route + migration déployées)
Latence API : <code>/health</code> ~1s, <code>/products</code> ~1s	Latence fixe Railway (infra), base ~0.05-0.1s

7. Notes de déploiement

- Railway applique automatiquement `prisma migrate deploy` au déploiement → la migration `20260814002635` s'applique seule, sans intervention.
- **Ne pas relancer `npm run db:seed`** en production (idempotent mais lent).
- Frontend associé : branche `Frontend` (boutons « Reprendre la vente », « Contacter le support » avec tag annonce/vendeur/commande, bandeau mise à jour PWA, fixes socket).